

Buku Ajar: Pemrograman Game Menggunakan Unity

Tim Pengajar Program Studi Teknik Informatika

Fakultas Teknologi Informasi

Universitas Bale Bandung

9 Oktober 2025

Daftar Isi

1	Pengenalan Pengembangan Game dan Unity	3
1.1	Konsep Dasar Game	3
1.2	Genre Game	3
1.3	Antarmuka dan Proyek Unity	4
2	Dasar Pemrograman C# di Unity	5
2.1	Struktur Skrip C#	5
2.2	Fungsi <code>Start()</code> dan <code>Update()</code>	5
2.3	Variabel dan Tipe Data	6
3	GameObject dan Komponen Unity	7
3.1	Transform	7
3.2	Rigidbody dan Collider	7
3.3	Prefab	8
4	Manajemen Aset dan Scene	9
4.1	Impor Aset	9
4.2	Pengaturan Scene	9
4.3	Pencahayaannya Dasar	10
5	Minggu 5: Kontrol Pemain dan Input	11
5.1	Pemrosesan Input	11
5.2	Menggerakkan Objek	11
6	Minggu 6: Sistem Fisika	13
6.1	Simulasi Fisika dengan Rigidbody	13
6.2	Tumbukan dan Material Fisika	13
7	Minggu 7: Animasi Game	15
7.1	Animator dan Animator Controller	15
7.2	Animasi Sprite dan 3D	15
8	Minggu 8: Antarmuka Pengguna (UI)	17
8.1	Kanvas dan Elemen UI	17

8.2	Membangun HUD	17
9	Minggu 9: Audio dalam Unity	19
9.1	Sistem Audio	19
9.2	Musik dan Efek Suara	19
10	Minggu 10: Kecerdasan Buatan (AI) dasar	21
10.1	Agan Sederhana dan Navigasi	21
10.2	Skrip Perilaku Dasar	21
11	Minggu 11: Mekanik Permainan	23
11.1	Sistem Skor dan Progres	23
11.2	Kondisi Menang/Kalah dan State Machine	23

Informasi Program Studi

Program Studi Teknik Informatika, Fakultas Teknologi Informasi, Universitas Bale Bandung. Mata kuliah ini membahas pemrograman game menggunakan Unity dengan fokus pada praktik rekayasa perangkat lunak permainan. Buku ajar ini dirancang sebagai pendamping perkuliahan dan bahan belajar mandiri.

Buku disusun agar dapat dikompilasi per-bab dan sebagai satu buku utuh. Setiap bab memuat tujuan pembelajaran, materi pokok, dan rujukan sumber terbuka. Struktur bab mengikuti rencana pembelajaran yang telah ditetapkan pada dokumen RPS.

Bab 1

Pengenalan Pengembangan Game dan Unity

1.1 Konsep Dasar Game

Game adalah sistem interaktif dengan aturan, tujuan, dan umpan balik yang membentuk pengalaman bermain. Dalam pengembangan perangkat lunak, game merepresentasikan dunia simulatif yang berubah sebagai respons terhadap input dan logika yang dirancang, sehingga interoperasi antara mekanika, antarmuka, dan aset menjadi krusial. Perancangan loop inti (core loop)—misalnya siklus mengeksplorasi, beraksi, dan menerima umpan balik—menjadi fondasi keterlibatan pemain yang berkelanjutan (Nystrom 2014; Fullerton 2008).

Mesin permainan (game engine) menyediakan abstraksi grafis, audio, input, fisika, dan manajemen aset agar tim berfokus pada konten dan mekanika alih-alih subsistem rendah-level. Pemilihan engine memengaruhi alur kerja, kinerja, dan portabilitas produk. Unity banyak dipakai di industri dan pendidikan karena ekosistemnya luas, dokumentasi komprehensif, dan dukungan lintas platform yang matang (*Unity Manual: Introduction to Unity* 2024; *Unity Scripting API: MonoBehaviour.Start, Update* 2024).

1.2 Genre Game

Genre menyediakan kerangka untuk mengelompokkan mekanika, ritme permainan, dan ekspektasi pengguna sehingga tim dapat memprioritaskan fitur inti sejak awal. Klasifikasi genre membantu menentukan kebutuhan kamera, antarmuka, dan kontrol interaksi, misalnya perbedaan signifikan antara platformer 2D dan RPG berbasis aksi. Dengan basis ini, perencanaan teknis dan desain dapat disinkronkan sejak tahap praproduksi (Schell 2019).

Walau genre deskriptif, inovasi mekanika tetap dimungkinkan dengan menggabungkan

elemen dari berbagai genre secara terkontrol. Strategi yang efektif ialah memetakan fitur genre ke pola implementasi yang dapat digunakan kembali, seperti objek kolektibel, sistem inventori, atau percabangan misi. Pendekatan berbasis pola memudahkan dokumentasi dan mengurangi risiko teknis saat iterasi (Nystrom 2014; Schell 2019).

Tabel 1.1: Contoh pemetaan genre ke fokus teknis

Genre	Fokus Mekanika	Implikasi Teknis
Platformer 2D	Lompatan presisi, rintangan	Fisika 2D, deteksi tumbukan, kamera sisi
RPG Aksi	Pertarungan, progres karakter	Sistem status, AI dasar, antarmuka inventori
Puzzle	Tantangan logika	State-space kecil, UI instruktif, telemetri kesulitan

1.3 Antarmuka dan Proyek Unity

Editor Unity terdiri atas panel utama seperti *Hierarchy*, *Scene*, *Inspector*, *Project*, dan *Console* yang masing-masing melayani tugas berbeda. *Hierarchy* mengelola struktur objek, *Scene* memberi konteks visual, *Inspector* menyunting properti, *Project* mengatur aset, dan *Console* menampilkan pesan sistem. Pemahaman fungsi tiap panel mempercepat alur kerja dan mengurangi kesalahan konfigurasi saat membangun fitur permainan (*Unity Manual: Introduction to Unity* 2024).

Struktur proyek Unity lazim meliputi *Assets*, *Packages*, dan *ProjectSettings* dengan penataan aset yang konsisten untuk kolaborasi. Praktik umum adalah memisahkan *Art*, *Audio*, *Prefabs*, dan *Scenes*, serta mendokumentasikan konvensi impor dan penamaan. Mengikuti struktur yang direkomendasikan dokumentasi resmi membantu menjaga kinerja editor dan kompatibilitas lintas versi (*Unity Manual: Project structure and files* 2024).



Gambar 1.1: Core loop sederhana: eksplorasi, aksi, dan umpan balik (Nystrom 2014).

Bab 2

Dasar Pemrograman C# di Unity

2.1 Struktur Skrip C#

Skrip di Unity umumnya berupa kelas C# yang mewarisi `MonoBehaviour` lalu dipasang pada *GameObject*. Struktur ini membuat skrip berpartisipasi dalam pesan siklus hidup (*lifecycle messages*) yang dipanggil mesin Unity seperti `Awake()`, `Start()`, dan `Update()`, serta berinteraksi dengan komponen lain di adegan. Konvensi penamaan yang konsisten dan pembagian tanggung jawab yang jelas antarkomponen meningkatkan keterbacaan dan pemeliharaan (*C# Programming Guide* 2024; *Unity Manual: Execution Order of Event Functions* 2024).

Setiap berkas skrip lazimnya memuat *namespace*, deklarasi kelas, bidang (field), dan metode dengan antarmuka publik yang minimal. Praktik yang disarankan adalah menghindari logika bercabang berlebihan di satu kelas, dan memecah perilaku ke komponen kohesif melalui komposisi. Dengan pendekatan ini, proses pengujian, penggantian komponen, dan kolaborasi tim menjadi lebih terstruktur dan dapat diprediksi (*C# Programming Guide* 2024).

2.2 Fungsi `Start()` dan `Update()`

Unity memanggil `Start()` sekali ketika komponen diinisialisasi secara aktif, dan `Update()` pada setiap frame saat komponen diaktifkan. Selain itu, `Awake()` terjadi lebih awal untuk menyiapkan dependensi, `OnEnable()` dipanggil saat komponen diaktifkan, dan `LateUpdate()` cocok untuk penyesuaian pasca-`Update()` seperti kamera. `FixedUpdate()` digunakan untuk logika fisika pada interval tetap agar simulasi stabil (*Unity Scripting API: MonoBehaviour.Start, Update* 2024; *Unity Manual: Execution Order of Event Functions* 2024).

Desain yang baik menempatkan inisialisasi yang berat di `Awake()` atau `Start()` dan meminimalkan pekerjaan tiap frame di `Update()`. Untuk mengurangi beban CPU, gu-

nakan *early return*, penjadwalan ulang proses, dan *coroutines* saat menunggu peristiwa atau waktu, sehingga alur tetap responsif tanpa *busy waiting*. Pemisahan logika peristiwa dan status juga membantu mencegah kompleksitas tak terkelola (*Unity Scripting API: MonoBehaviour.Start, Update* 2024; *Unity Manual: Coroutines* 2024).

Tabel 2.1: Ringkasan event lifecycle pada `MonoBehaviour` (*Unity Manual: Execution Order of Event Functions* 2024)

Event	Tujuan singkat
<code>Awake()</code>	Inisialisasi awal sebelum referensi antarkomponen siap, dipanggil sekali
<code>OnEnable()</code>	Dipanggil saat komponen diaktifkan atau diaktifkan kembali
<code>Start()</code>	Inisialisasi setelah semua <code>Awake()</code> , dipanggil sekali
<code>Update()</code>	Logika per frame, bergantung laju bingkai
<code>FixedUpdate()</code>	Logika fisika pada interval tetap
<code>LateUpdate()</code>	Penyesuaian setelah <code>Update()</code> , misalnya kamera
<code>OnDisable()</code>	Dipanggil saat komponen dinonaktifkan
<code>OnDestroy()</code>	Pelepasan sumber daya sebelum objek dihancurkan

2.3 Variabel dan Tipe Data

C# menyediakan tipe nilai dan tipe referensi untuk menyatakan keadaan objek permainan. Pemilihan tipe yang tepat—misalnya `float` untuk koordinat dan `Vector3` dari API Unity untuk posisi—meningkatkan kejelasan maksud dan mengurangi kebutuhan konversi. Selain itu, modifikator akses seperti `public` dan `private` mengatur enkapsulasi sehingga antarkomponen berinteraksi melalui antarmuka yang jelas (*C# Programming Guide* 2024; *C# types and variables* 2024).

Di Unity, bidang bertanda `[SerializeField]` atau bertipe `public` dapat diedit melalui *Inspector*, memudahkan penyetelan tanpa mengubah kode. Pemahaman aturan serialisasi Unity—termasuk dukungan terhadap tipe tertentu dan perilaku referensi—membantu menghindari kejutan saat memuat adegan. Untuk data bersama lintas adegan, pola *data-as-asset* menggunakan `ScriptableObject` dapat dipertimbangkan (*Unity Manual: Script serialization* 2024; *Unity Manual: ScriptableObject* 2024).

Bab 3

GameObject dan Komponen Unity

3.1 Transform

Setiap *GameObject* memiliki komponen **Transform** yang menyimpan posisi, rotasi, dan skala dalam ruang dunia atau lokal. **Transform** juga menentukan hierarki orang tua–anak yang memengaruhi sistem koordinat, sehingga transformasi pada induk ikut memengaruhi anak. Pemahaman relatif posisi lokal dan dunia membantu mencegah perilaku tak terduga saat menerapkan animasi dan fisika (*Unity Scripting API: Transform* 2024; *Unity Manual: GameObjects* 2024).

Operasi **Transform** sebaiknya mempertimbangkan *frame rate* dan integrasi fisika agar tidak terjadi ketidakselarasan. Untuk pergerakan berbasis fisika, gunakan gaya atau kecepatan pada **Rigidbody** dan hindari mengubah **Transform** langsung, kecuali untuk penempatan awal. Praktik ini menjaga konsistensi simulasi tumbukan dan gravitasi (*Unity Scripting API: Transform* 2024).

3.2 Rigidbody dan Collider

Rigidbody menambahkan dinamika fisika pada objek, memungkinkan penerapan gaya, torsi, dan pengaruh gravitasi. Bersama **Collider**, sistem ini mendukung pendeteksian tumbukan dan respons fisik sesuai material. Penyetelan massa, drag, dan kendala perlu seimbang untuk stabilitas, sementara mode deteksi tumbukan *continuous* mencegah objek cepat menembus permukaan (*Unity Scripting API: Rigidbody* 2024; *Unity Scripting API: Collider* 2024).

Perbedaan *dynamic* dan *kinematic* **Rigidbody** berdampak pada interaksi: objek kinematik digerakkan skrip namun tetap dapat memicu tumbukan. **Collider** bertipe *trigger* cocok untuk deteksi kawasan tanpa respons fisika, sedangkan *collider non-trigger* untuk tumbukan nyata. Pemilihan kombinasi harus diselaraskan dengan tujuan desain interaksi (*Unity Scripting API: Collider* 2024).

Tabel 3.1: Perbandingan tipe **Rigidbody** dan penggunaan collider

Tipe	Karakteristik	Kasus Penggunaan
Dynamic	Dipengaruhi gaya dan gravitasi	Objek fisik interaktif (peti, proyektil)
Kinematic	Dikendalikan skrip, tidak dipengaruhi gaya	Pintu geser, platform bergerak
Trigger Collider	Tidak ada respons fisika, memicu event	Zona pickup, deteksi area

3.3 Prefab

Prefab menyediakan cetak biru objek untuk instansiasi ulang yang konsisten di berbagai adegan. Dengan menyimpan konfigurasi komponen sebagai aset, pembaruan pada definisi utama akan mengalir ke seluruh instans. Fitur varian dan override memungkinkan kustomisasi sambil mempertahankan hubungan ke definisi dasar (*Unity Manual: Prefabs* 2024).

Arsitektur berbasis prefab mendorong modularitas dan pengujian. Pemisahan logika, data, dan presentasi mengurangi dampak perubahan silang, sementara konvensi penamaan dan struktur folder yang rapi mempercepat pencarian aset. Dokumentasi pola penggunaan prefab di tim membantu mengurangi regresi saat iterasi (*Unity Manual: Prefabs* 2024; *Unity Manual: Project structure and files* 2024).



Gambar 3.1: Relasi prefab dasar, varian, dan instans di scene (*Unity Manual: Prefabs* 2024).

Bab 4

Manajemen Aset dan Scene

4.1 Impor Aset

Unity mendukung impor beragam format aset seperti gambar, audio, dan model 3D, masing-masing dengan pengaturan impor yang memengaruhi kualitas dan kinerja. Untuk tekstur, opsi seperti resolusi, kompresi, dan format (mis. ASTC/ETC) berdampak pada kualitas visual dan ukuran unduhan; untuk audio, pilihan kompresi dan streaming memengaruhi latensi dan memori. Pengaturan yang tepat menyeimbangkan kualitas dengan runtime dan target platform (*Unity Manual: Importing assets* 2024; *Unity Manual: Platform-specific texture overrides* 2024).

Pengembang dianjurkan menetapkan preset impor agar konsisten di seluruh proyek, terutama dalam tim besar. Gunakan konvensi penamaan dan struktur folder berdasarkan tipe aset untuk mempercepat integrasi dan meminimalkan konflik kontrol versi. Dokumentasikan konfigurasi impor dan pipeline sumber sehingga pembaruan dari vendor eksternal tetap konsisten (*Unity Manual: Project structure and files* 2024; *Unity Manual: Importing assets* 2024).

4.2 Pengaturan Scene

Scene merepresentasikan susunan objek permainan pada konteks tertentu, seperti level atau menu utama. Pembagian fitur ke beberapa scene memudahkan pemuatan bertahap dan pengujian terisolasi, sekaligus mengurangi waktu iterasi. Manajemen ketergantungan antar scene menjadi penting agar transisi mulus dan bebas kebocoran sumber daya (*Unity Manual: Scenes* 2024).

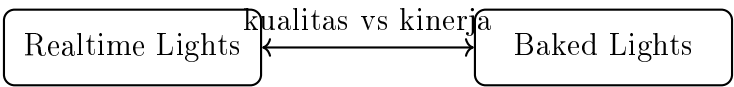
Additive scene loading memungkinkan beberapa scene aktif bersamaan, misalnya memisahkan geometri statis dari UI atau sistem gameplay. Pendekatan ini mendukung organisasi kerja lintas disiplin dan mengurangi konflik saat kontrol versi. Gunakan Build Settings dan indeks scene untuk menjamin urutan pemuatan yang konsisten (*Unity Ma-*

nual: *Scenes* 2024).

Tabel 4.1: Strategi organisasi scene		
Strategi	Kelebihan	Pertimbangan
Satu scene besar	Sederhana untuk prototipe	Waktu muat panjang, risiko konflik versi
Multi-scene additive	Paralel lintas disiplin, modular	Butuh manajemen dependensi dan urutan load
Scene per sistem (UI/level)	Isolasi yang jelas	Koordinasi transisi antar scene

4.3 Pencahayaan Dasar

Sistem pencahayaan menentukan persepsi kedalaman, suasana, dan keterbacaan komposisi visual. Unity menyediakan *realtime lighting* untuk sumber dinamis dan *baked lighting* untuk optimasi pada geometri statis; kombinasi yang tepat meningkatkan kinerja tanpa menurunkan kualitas. Kalibrasi intensitas, warna, dan *lightmap* perlu mempertimbangkan target perangkat keras (*Unity Manual: Lighting* 2024).



Gambar 4.1: Spektrum pengaturan pencahayaan antara realtime dan baked (*Unity Manual: Lighting* 2024).

Penggunaan *reflection probes* dan *light probes* meningkatkan realisme pada objek bergerak tanpa biaya komputasi yang berlebihan. Penempatan dan ukuran volume harus mempertimbangkan material dominan, jarak kamera, dan dinamika adegan. Praktik ini menjaga konsistensi visual lintas platform (*Unity Manual: Lighting* 2024).

Bab 5

Minggu 5: Kontrol Pemain dan Input

5.1 Pemrosesan Input

Unity menyediakan dua pendekatan utama untuk input: sistem lama (*Legacy Input*) dan paket *Input System* yang lebih modern. Keduanya memungkinkan pembacaan perangkat seperti keyboard, mouse, dan pengendali permainan, namun *Input System* menawarkan pemetaan aksi yang lebih terstruktur. Pemilihan sistem hendaknya mempertimbangkan kebutuhan proyek, kompatibilitas platform, dan kemudahan pemeliharaan (*Unity Scripting API: Input* 2024; *Unity Manual: Input System* 2024).

Pemetaan input berbasis aksi memisahkan niat pemain (misalnya ?melompat?) dari perangkat fisik yang digunakan. Hal ini memudahkan dukungan multi-perangkat dan aksesibilitas, sekaligus menyederhanakan pengujian. Dokumentasi skema input yang konsisten mempercepat adaptasi anggota tim baru dan mengurangi regresi saat menambah kontrol baru (*Unity Manual: Input System* 2024).

5.2 Menggerakkan Objek

Pergerakan karakter atau objek dapat diimplementasikan melalui perubahan **Transform**, penerapan gaya pada **Rigidbody**, atau penggunaan karakter kontroler. Setiap pendekatan memiliki implikasi pada responsivitas dan konsistensi fisika yang dirasakan. Pemilihan metode harus diselaraskan dengan genre dan tujuan desain (*Unity Scripting API: Rigidbody* 2024; *Unity Scripting API: Transform* 2024).

Untuk menjaga stabilitas simulasi, gunakan **FixedUpdate()** ketika menggerakkan objek dengan fisika dan terapkan normalisasi vektor serta pembatasan kecepatan jika perlu. Teknik *lerp* dan *smooth damping* dapat meningkatkan rasa halus pada peralihan gerakan. Pengujian pada berbagai laju bingkai diperlukan untuk memastikan perilaku yang konsisten lintas perangkat (*Unity Scripting API: Transform* 2024).

Bab 6

Minggu 6: Sistem Fisika

6.1 Simulasi Fisika dengan Rigidbody

Rigidbody memungkinkan objek mengikuti hukum dinamika Newton kira-kira dengan integrasi numerik yang efisien. Parameter seperti massa, drag, dan gravitasi menentukan respons terhadap gaya dan torsi. Penggunaan metode `AddForce` dan `AddTorque` memberi kontrol yang presisi terhadap akselerasi dan momentum (*Unity Scripting API: Rigidbody* 2024).

Pengaturan mode interpolasi dan deteksi tumbukan berpengaruh pada stabilitas serta akurasi. Untuk objek berkecepatan tinggi, gunakan *continuous collision detection* guna mencegah objek menembus permukaan. Penguncian sumbu (constraints) dapat menghindari gerakan yang tidak diinginkan dalam simulasi (*Unity Scripting API: Rigidbody* 2024).

6.2 Tumbukan dan Material Fisika

Kombinasi collider dan material fisika menentukan sifat kontak seperti gesekan dan pantulan. Penggunaan `PhysicMaterial` memudahkan konsistensi antar permukaan dalam permainan. Penyetelan yang hati-hati diperlukan agar pengalaman bermain tetap dapat diprediksi dan menyenangkan (*Unity Scripting API: Collider* 2024).

Peristiwa `OnCollisionEnter` dan `OnTriggerEnter` memberi sinyal terjadinya interaksi yang dapat dihubungkan ke logika permainan. Pemisahan logika deteksi dan respons mengurangi kompleksitas serta mempermudah pengujian unit. Penerapan *layers* dan masker tumbukan juga membantu mengatur kategori interaksi yang diperbolehkan (*Unity Scripting API: Collider* 2024).

Bab 7

Minggu 7: Animasi Game

7.1 Animator dan Animator Controller

Sistem Animator mengelola keadaan animasi melalui grafik keadaan (state machine) dan transisi. *Animator Controller* memetakan parameter ke peralihan antar keadaan sehingga animasi dapat merespons input dan logika permainan. Dengan desain parameter yang ringkas, kompleksitas transisi dapat ditekan tanpa mengorbankan ekspresivitas (*Unity Manual: Animator Controllers* 2024; *Unity Manual: State machines (Animator)* 2024).

Penggunaan lapisan (layers) dan penggabungan (blending) memungkinkan kombinasi animasi sebagian tubuh, seperti menjalankan animasi lari sambil mengayunkan senjata. Penyetelan kurva transisi dan waktu *exit* memengaruhi keluwesan dan ketepatan respons animasi. Dokumentasi graf keadaan membantu kolaborasi antara desainer dan programmer (*Unity Manual: Animator Controllers* 2024).

7.2 Animasi Sprite dan 3D

Untuk sprite 2D, klip animasi sering dibentuk dari rangkaian gambar atau *sprite sheet*, sedangkan pada 3D, klip animasi berasal dari data *rigging* dan *skinning*. Keduanya memanfaatkan *clips* dan *timelines* yang dapat disusun ulang untuk berbagai konteks. Konsistensi *frame rate* dan skala sangat penting untuk menghindari ketidakselarasan gerakan (*Unity Manual: Animation Clips* 2024).

Teknik *root motion* dapat digunakan untuk menyelaraskan perpindahan karakter di dunia dengan pergerakan pada klip, mengurangi selisih antara niat desain dan hasil simulasi. Validasi terhadap kolider dan navmesh diperlukan agar animasi tidak menyebabkan penetrasi geometri atau lompatan yang tidak realistis. Dengan siklus iterasi yang baik, kualitas presentasi dapat ditingkatkan tanpa biaya komputasi berlebihan (*Unity Manual: Animator Controllers* 2024).

Bab 8

Minggu 8: Antarmuka Pengguna (UI)

8.1 Kanvas dan Elemen UI

Unity menyediakan dua teknologi utama untuk UI, yakni UGUI dan UI Toolkit, yang masing-masing cocok untuk kebutuhan produksi yang berbeda. UGUI lazim digunakan untuk HUD dan menu interaktif, sementara UI Toolkit menawarkan arsitektur modern berbasis gaya dan markup. Pemilihan teknologi harus mempertimbangkan target platform, kebutuhan kinerja, dan alur kerja tim (*Unity Manual: UI Toolkit and UGUI* 2024).

Struktur kanvas memengaruhi performa karena perubahan tata letak dapat memicu perhitungan ulang yang mahal. Menjaga hierarki UI tetap dangkal dan meminimalkan penghitungan ulang tata letak adalah praktik yang direkomendasikan. Penggunaan *Sprite Atlas* dan *batching* juga membantu menjaga laju bingkai konsisten pada perangkat terbatas (*Unity Manual: UI Toolkit and UGUI* 2024).

8.2 Membangun HUD

HUD menyediakan informasi status yang kritikal seperti kesehatan, skor, dan navigasi. Desain yang baik mengutamakan keterbacaan, hierarki visual, dan responsivitas terhadap berbagai resolusi layar. Pengujian aksesibilitas—misalnya kontras warna dan ukuran tipografi—meningkatkan inklusivitas pengalaman bermain (*Unity Manual: UI Toolkit and UGUI* 2024).

Integrasi HUD dengan sistem permainan harus menghindari ketergantungan siklik dengan logika inti. Penerapan *event-driven* atau *data binding* mengurangi keterhubungan kuat dan memudahkan penggantian komponen. Pendekatan ini juga membuat UI lebih mudah diuji dan diperluas di masa depan (*Unity Manual: UI Toolkit and UGUI* 2024).

Bab 9

Minggu 9: Audio dalam Unity

9.1 Sistem Audio

Sistem audio Unity meliputi pemutaran klip, pengendalian volume, dan pemrosesan spasial. `AudioSource` bertanggung jawab atas pemutaran, sementara pengaturan proyek dan `AudioListener` menentukan bagaimana pemain menerima suara. Penempatan sumber audio dan pengaturan jarak turut memengaruhi persepsi kedalaman dan imersi (*Unity Manual: Audio* 2024; *Unity Scripting API: AudioSource* 2024).

Strategi manajemen audio yang efektif mencakup penggunaan mixer, grup, dan efek untuk menjaga konsistensi loudness di seluruh permainan. Penggunaan kompresi yang tepat pada aset audio membantu mengurangi ukuran gim tanpa mengorbankan kualitas yang berarti. Dokumentasi pengaturan audio mempermudah proses pascaproduksi dan porting (*Unity Manual: Audio* 2024).

9.2 Musik dan Efek Suara

Musik latar mengatur suasana, sementara efek suara memberikan umpan balik instan atas aksi pemain dan peristiwa permainan. Sinkronisasi musik dengan transisi adegan atau keadaan permainan memperkuat koherensi naratif dan ritme pengalaman. Pipeline *import* dan normalisasi level harus konsisten agar perbedaan dinamika tidak mengganggu pemain (*Unity Manual: Audio* 2024).

Penggunaan parameter dalam mixer atau skrip memungkinkan penyesuaian dinamis, seperti memudahkan musik saat dialog atau memperkuat efek saat momen penting. Pengujian pada berbagai perangkat memastikan hasil yang konsisten dan menghindari clipping atau distorsi. Praktik ini membantu menjaga kualitas produksi secara menyeluruh (*Unity Scripting API: AudioSource* 2024).

Bab 10

Minggu 10: Kecerdasan Buatan (AI) dasar

10.1 Agen Sederhana dan Navigasi

Penerapan AI dasar dalam permainan sering dimulai dari agen yang menavigasi lingkungan menuju target. Unity menyediakan sistem navigasi dan *pathfinding* berbasis NavMesh yang memudahkan perencanaan rute pada medan yang dapat dilalui. Dengan memanfaatkan NavMeshAgent, pengembang dapat mengatur kecepatan, akselerasi, dan penghindaran hambatan secara deklaratif (*Unity Manual: Navigation and pathfinding* 2024).

Desain perilaku harus mempertimbangkan kondisi lingkungan dinamis seperti pintu yang terbuka tutup atau objek bergerak. Pembaruan NavMesh sebagian (*carving*) dan pengendalian tujuan sementara dapat meningkatkan naturalitas gerak. Pengujian pada peta representatif mengurangi kejutan saat berpindah ke level yang lebih kompleks (*Unity Manual: Navigation and pathfinding* 2024).

10.2 Skrip Perilaku Dasar

Selain navigasi, agen membutuhkan logika pengambilan keputusan untuk memilih aksi berdasarkan keadaan. Pola *finite state machine* (FSM) lazim digunakan untuk mengelola peralihan antar keadaan seperti ?patroli?, ?mengejar?, dan ?kembali?. Pendekatan modular dengan antarmuka yang jelas memudahkan perluasan perilaku tanpa menambah hutang teknis (*Unity Manual: State machines (Animator)* 2024; Nystrom 2014).

Pemisahan antara deteksi (misalnya jarak dan garis pandang) dan tindakan (misalnya bergerak atau menyerang) membantu menjaga keterbacaan dan pengujian. Dokumentasi diagram FSM serta parameter transisi mempercepat sinkronisasi dengan tim desain. Praktik ini mengurangi risiko regresi ketika menambahkan keadaan baru di iterasi berikutnya (Nystrom 2014).

Bab 11

Minggu 11: Mekanik Permainan

11.1 Sistem Skor dan Progres

Sistem skor memberikan metrik kuantitatif atas performa pemain dan mendorong eksplorasi strategi yang lebih efektif. Implementasi yang baik memisahkan penyimpanan skor dari presentasi, sehingga perubahan tampilan tidak mempengaruhi logika inti. Penyimpanan persisten melalui *PlayerPrefs* atau sistem penyimpanan khusus memungkinkan retensi progres antar sesi bermain (*Unity Manual: Introduction to Unity* 2024).

Pengaturan ekonomi permainan seperti hadiah, penalti, dan *multiplier* harus dirancang untuk menjaga keseimbangan tantangan. Telemetri desain—misalnya distribusi skor dan tingkat penyelesaian—membantu penyesuaian parameter tanpa menebak-nebak. Dengan cara ini, pengalaman bermain dapat dioptimalkan secara bertahap berdasarkan data (Nystrom 2014).

11.2 Kondisi Menang/Kalah dan State Machine

Penentuan kondisi akhir memberikan sasaran yang jelas dan kerangka evaluasi bagi pemain. Struktur *state machine* membantu mengatur transisi antara keadaan seperti ?bermain?, ?jeda?, ?menang?, dan ?kalah?. Dengan membatasi efek samping pada saat transisi, risiko perilaku tak terduga dapat dikurangi (*Unity Manual: State machines (Animator)* 2024).

Pemodelan keadaan yang eksplisit juga memudahkan integrasi dengan UI dan audio, misalnya memicu animasi kemenangan atau musik latar tertentu. Pengujian jalur kritis—termasuk skenario tepi seperti rematch atau memuat ulang—diperlukan untuk memastikan stabilitas. Praktik dokumentasi keadaan dan transisi menjaga kejelasan arsitektur sepanjang siklus hidup proyek (*Unity Manual: State machines (Animator)* 2024; Nystrom 2014).

Bibliografi

- C# Programming Guide* (2024). Diakses 2025-10-09. URL: <https://learn.microsoft.com/dotnet/csharp/programming-guide/>.
- C# types and variables* (2024). Diakses 2025-10-09. URL: <https://learn.microsoft.com/dotnet/csharp/language-reference/builtin-types/built-in-types>.
- Fullerton, Tracy (2008). *Game Design Workshop: A Playcentric Approach to Creating Innovative Games*. Open access excerpt, Diakses 2025-10-09. CRC Press. URL: https://www.designersnotebook.com/Books/Fullerton_Second_Edition.pdf.
- Nystrom, Robert (2014). *Game Programming Patterns*. Open access, Diakses 2025-10-09. URL: <https://gameprogrammingpatterns.com/>.
- Schell, Jesse (2019). *The Art of Game Design: A Book of Lenses*. 3 edition. Supplementary materials available online, Diakses 2025-10-09. CRC Press. URL: <https://www.crcpress.com/The-Art-of-Game-Design-A-Book-of-Lenses/Schell/p/book/9781138632059>.
- Unity Manual: Animation Clips* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/AnimationClips.html>.
- Unity Manual: Animator Controllers* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/Animator.html>.
- Unity Manual: Audio* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/Audio.html>.
- Unity Manual: Coroutines* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/Coroutines.html>.
- Unity Manual: Execution Order of Event Functions* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/ExecutionOrder.html>.
- Unity Manual: GameObjects* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/GameObjects.html>.
- Unity Manual: Importing assets* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/ImportingAssets.html>.
- Unity Manual: Input System* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.7/manual/>.
- Unity Manual: Introduction to Unity* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/index.html>.

- Unity Manual: Lighting* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/Lighting.html>.
- Unity Manual: Navigation and pathfinding* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/Navigation.html>.
- Unity Manual: Platform-specific texture overrides* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/class-TextureImporterOverride.html>.
- Unity Manual: Prefabs* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/Prefabs.html>.
- Unity Manual: Project structure and files* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/StructureOfAProject.html>.
- Unity Manual: Scenes* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/CreatingScenes.html>.
- Unity Manual: Script serialization* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/script-Serialization.html>.
- Unity Manual: ScriptableObject* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/class-ScriptableObject.html>.
- Unity Manual: State machines (Animator)* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/StateMachineBasics.html>.
- Unity Manual: UI Toolkit and UGUI* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/Manual/UnityUI.html>.
- Unity Scripting API: AudioSource* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/ScriptReference/AudioSource.html>.
- Unity Scripting API: Collider* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/ScriptReference/Collider.html>.
- Unity Scripting API: Input* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/ScriptReference/Input.html>.
- Unity Scripting API: MonoBehaviour.Start, Update* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>.
- Unity Scripting API: Rigidbody* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/ScriptReference/Rigidbody.html>.
- Unity Scripting API: Transform* (2024). Diakses 2025-10-09. URL: <https://docs.unity3d.com/ScriptReference/Transform.html>.